ELEC 328 Assignment 3: Particle Worlds

John Tucker – jrt15

Screenshots: 328 Assignment 3 Just Screenshots
The Unity project is at the root of the repository (github.com/johnjrtuck/john-tucker-elec328-f26-assn3) and the screenshots pdf and excel sheet should be in the deliverables.
Results table: 328 Assignment 3 Stats

Setup:

- Unity 6.6 (6000.6.0f1), DX12
- URP with the PC_RP asset
- Game view at 1920×1080
- VSync set to Don't Sync
- all measurements taken in Editor Play mode
- the Roost PC's hardware

Test setup and friend session

I did the session with Emma. We took turns and I went first. One of us watched the particles in the headset at the Roost while the other changed the settings in Unity. We had issue getting the project to run because of version differences but after a while we were able to get it to work through trial and error. Once we got it working the test was pretty intuitive. I used the sparks particles to test all the different settings. I say that but we actually had to redo it a few different times to make sure the videos were working and all the graphics information was actually on screen the entire time. We had a lot of issues getting motion sickness from all the particles especially when the entity count got super high but in the end I think we got all the data correct.

Rendering and Population:

At 100, 1,000 and 5,000 particles, GameObject mode produced 221, 1,859 and 10,051 draw calls (5, 4, 6), which works out to roughly two per particle, and nearly all of them were handled by the SRP Batcher. Instanced mode never went above 81 draw calls and ranged from 23 to 81 (1, 2, 3), even at 5,000 particles with about 20,000 mesh instances, since Standard Instanced rendering sends a whole batch of identical meshes to the GPU in one call. SetPass calls rose much sloweR than draw calls in GameObject mode, only 38 to 66, because the SRP Batcher keeps the same material bound across consecutive draws instead of switching state each time. Where the CPU cost really showed up was the main thread vs render thread split. In GameObject mode the main thread's share climbed from 21% to 28% to 40% (5, 4, 6), while in instanced mode it stayed between 17% and 25% (1, 2, 3). I'M pretty sure that extra load comes from the CPU having to update and hand off thousands of individual Transforms every frame,

and that the particle math is about the same in both modes. Frame time held between 11.1 and 13.8 ms (roughly 73 to 90 FPS) across all six trials, and GPU time stayed at or below 1.4 ms. Since the frame was capped by the 90 FPS headset sync, i believe the extra CPU work was hidden in the frame time. the draw call counts and thread load are where the difference shows. The profiler had a lot of spikes, so I only took readings during the steady stretches between them.

Dynamics

For the dynamics test I kept instanced mode with 1,000 particles and MSAA 8x, and only changed Simulation Speed. I started at the baseline of 1 (1), then tried 2 (7) and 0.2 (8). Performance barely moved. The baseline frame time was 13.8 ms, and both the 2x and 0.2x runs came in at 12.3 ms (about 81 FPS). GPU time stayed between 0.6 and 0.8 ms, and the main thread share went from 25.4% at baseline to 25.4% at 2x and 22.0% at 0.2x. That makes sense to me. The benchmark updates the same 1,000 particles every frame no matter the speed, so speed only changes how far each one moves per frame, not how much work is done. The draw calls and SetPass calls were lower at baseline (23 and 15) than at 2x and 0.2x (57/38 and 58/39). The 2x and 0.2x numbers almost match each other, so I think the difference comes from where the camera was looking in the Roost, not from the speed change. Where speed did matter was comfort. At 2x the sparks felt intense to watch in the headset, and we both ended up a bit motion sick by the end, mostly when lots of particles were moving fast. I much preferred the look and how my head felt when doing the 0.2x speed. I think if I were to implement particles in the future I'd keep that in mind because in the headset I had to resist closing my eyes.

Anti-Aliasing

For anti-aliasing I kept the baseline setup (instanced, 1,000 particles, speed 1) and changed the MSAA setting on the URP asset between disabled (9), 2x (10), 4x (11) and 8x (12). Everything else in the session ran smoother at 8x, so 8x was my starting point. I prefer the 8x in general I felt like it made mine and Emma's heads hurt less. The Inspector in each screenshot shows the setting actually changed. Performance didn't really respond to it. Frame time was 11.3, 12.3, 12.8 and 11.1 ms (about 78 to 90 FPS), and GPU time stayed between 0.6 and 0.8 ms at every setting, with no upward trend at 8x. Since MSAA only adds extra samples along triangle edges, it mainly just makes the GPUwork harder. With GPU time already under 1 ms and the frame capped at the 90 FPS headset sync, there wasn't enough load for it to show up. Draw calls (56 to 72) and SetPass calls (36 to 47) shifted a little between runs, but I think that follows where the view was pointing, not the MSAA level, since MSAA doesn't change how many draws get sent. Visually there was a difference but it was easiest to see when comparing disabled vs 8x Emma said the edges looked better and smoother as the MSAA went up.

Preferred Set Up:

I'd go with instanced rendering, somewhere between 1,000 and 5,000 particles, at 0.2x speed with 8x MSAA. The slow speed honestly just looks really cool (T8, 8). It's way easier to watch for a long time than 2x, which got intense fast. Performance-wise either end was fine, since instanced held around 90 FPS the whole way (T1, T3). I'd keep it instanced, because GameObject mode piled up over 10,000 draw calls at 5,000 particles (T6). For anti-aliasing I'd pick 8x because it was the smoothest looking, and it didn't seem to cost much in frame time (T12). That is what I saved for my project doing. It should open at 1000 particles, 0.2x speed and with 8x MSAA instanced. I already uploaded my excel sheet and it might have said I chose 5000 particles but really I did 1000.

AI Policy:
AI was not used to write, rewrite, summarize, or substantially edit project documentation, including the report, technical explanation, reflection, screenshot annotations/captions, or written analysis in the results table. Record measurements were from the roost lab and screenshots were taken on the computer there.

On my honor, I have neither given nor received any unauthorized aid on this assignment
John Tucker